

Conversiono

Epic 3: JavaScript SDK

Browser-Side Behavioral Event Capture

CONV-28 through CONV-35

-
- Ch 1: SDK Architecture & Build Pipeline
 - Ch 2: TypeScript Types & Configuration
 - Ch 3: Cryptographic Hashing & Utilities
 - Ch 4: HTTP Transport & sendBeacon
 - Ch 5: Event Queue & Batching Strategy
 - Ch 6: Session Management & Persistence
 - Ch 7: DOM Event Collectors
 - Ch 8: Exit Intent & Visibility Detection
 - Ch 9: Backend API Endpoints (FastAPI)
 - Ch 10: Testing Strategy (Vitest + Pytest)

Chapter 1

SDK Architecture & Build Pipeline

1.1 Why a Custom SDK?

Third-party analytics tools (Google Analytics, Hotjar) send data to their own servers. Conversiono needs raw behavioral events flowing directly into our ML pipeline, with sub-second granularity on mouse velocity, scroll retreat patterns, and exit intent signals. A custom SDK gives us full control over what data is captured, how it is batched, and where it is sent.

1.2 Project Structure

```
sdk/  
  src/  
    index.ts      -- Public API: init(), destroy(), reportOutcome()  
    types.ts      -- TypeScript interfaces (RawEvent, Config, etc.)  
    utils.ts      -- sha256, uuid4, throttle, getElementId  
    transport.ts   -- HTTP fetch + sendBeacon for API calls  
    event-queue.ts -- Buffered event queue with periodic flush  
    session.ts     -- Session lifecycle + sessionStorage  
    collectors.ts  -- DOM event listeners (mouse, click, scroll, etc.)  
  tests/          -- Vitest test suite (74 tests)  
  dist/           -- Build output (UMD + ESM + .d.ts)  
  rollup.config.mjs  
  tsconfig.json  
  vitest.config.ts
```

1.3 Build Pipeline with Rollup

Rollup is a module bundler optimized for libraries (vs Webpack which targets apps). We produce three outputs from a single entry point:

1. UMD (conversiono.umd.js) - works with `<script>` tags, AMD, and CommonJS
2. ESM (conversiono.esm.js) - for modern bundlers (Vite, Webpack 5, esbuild)
3. TypeScript declarations (index.d.ts) - enables type-safe integration

The UMD build uses Terser for minification and exposes a global 'Conversiono' object so merchants can use it with a simple `<script>` tag without any build tooling.

```
// rollup.config.mjs (simplified)
export default [
  {
    input: "src/index.ts",
    output: {
      file: "dist/conversiono.umd.js",
      format: "umd",
      name: "Conversiono", // window.Conversiono
    },
    plugins: [typescript(), resolve(), terser()],
  },
  // ... ESM and DTS outputs
];
```

Key Concept: UMD vs ESM

UMD (Universal Module Definition) wraps your code so it works everywhere: browser globals, AMD (RequireJS), and CommonJS (Node). ESM (ES Modules) is the modern standard with static imports/exports that enable tree-shaking.

Chapter 2

TypeScript Types & Configuration

2.1 The Type System as Documentation

TypeScript types serve as living documentation. When a merchant integrates our SDK, their IDE shows exactly what options are available and what each event looks like. This is why we define strict interfaces rather than using 'any'.

Event Types (matching database constraints)

```
// types.ts
export type EventType =
  | "mouse_move" | "click" | "scroll"
  | "exit_intent" | "visibility";

export interface RawEvent {
  type: EventType;
  ts: string; // ISO 8601
  x?: number | null;
  y?: number | null;
  velocity?: number | null;
  element_id?: string | null;
  metadata?: Record<string, unknown> | null;
}
```

2.2 Configuration with Sensible Defaults

The SDK exposes a `ConversionoConfig` interface with only one required field (`sdkKey`). Everything else has production-ready defaults. Internally we 'resolve' the config into a `ResolvedConfig` where all fields are required.

```
// User provides (partial):
{ sdkKey: 'sk_live_abc123' }

// SDK resolves (complete):
{
  sdkKey: 'sk_live_abc123',
  sdkKeyHash: 'a1b2c3...', // SHA-256
  apiUrl: 'https://shop.com',
  flushIntervalMs: 2000,
  maxBatchSize: 50,
  mouseMoveThrottleMs: 100,
  debug: false,
  disabled: false,
}
```

Design Pattern: Partial -> Resolved Config

This pattern (user provides partial, SDK fills defaults) is standard in production SDKs. It keeps the API simple for merchants while giving the internal code guaranteed types. The nullish coalescing (??) operator handles the default assignment cleanly.

Chapter 3

Cryptographic Hashing & Utilities

3.1 Why Hash the SDK Key?

The merchant's SDK key is a secret. Sending it in plaintext over the wire would expose it to anyone inspecting network traffic. Instead, we SHA-256 hash it client-side and send only the hash. The backend stores the same hash (computed at key creation time) and matches them.

This is the same principle as password hashing: the secret never leaves the client in its original form. Even if someone intercepts the hash, they cannot reverse it to get the original key.

3.2 Web Crypto API

```
export async function sha256(message: string): Promise<string> {  
  // Use Web Crypto API (available in all modern browsers)  
  const encoder = new TextEncoder();  
  const data = encoder.encode(message);  
  const hash = await crypto.subtle.digest('SHA-256', data);  
  return Array.from(new Uint8Array(hash))  
    .map(b => b.toString(16).padStart(2, '0'))  
    .join('');  
}
```

The Web Crypto API is the browser's built-in cryptographic toolkit. It returns an `ArrayBuffer`, which we convert to a hex string. The function is `async` because `crypto.subtle.digest()` returns a `Promise` (the browser may use hardware acceleration).

3.3 The Throttle Function

Mouse move events fire 60+ times per second. Sending all of them would flood the queue and waste bandwidth. Throttling ensures we capture at most one event per N milliseconds while still capturing the final position.

```
export function throttle(fn, ms) {
  let last = 0;
  let timer = null;
  return function(...args) {
    const now = Date.now();
    const remaining = ms - (now - last);
    if (remaining <= 0) {
      // Enough time passed - fire immediately
      clearTimeout(timer); timer = null;
      last = now;
      fn.apply(this, args);
    } else if (!timer) {
      // Schedule trailing call
      timer = setTimeout(() => { ... }, remaining);
    }
  };
}
```

Throttle vs Debounce

Throttle: fires at regular intervals during activity. Use for continuous events like mousemove, scroll, resize.

Debounce: fires only AFTER activity stops. Use for discrete events like search input, window resize-end.

We use throttle because we need periodic samples DURING mouse movement.

Chapter 4

HTTP Transport & sendBeacon

4.1 The Transport Class

The Transport class is the SDK's HTTP client. It wraps `fetch()` for normal requests and `navigator.sendBeacon()` for page unload scenarios. All requests include the X-SDK-Key header with the hashed key.

API Endpoints

- POST `/api/v1/sessions` - Create a new tracking session
- PUT `/api/v1/sessions/:id/end` - End a session
- PUT `/api/v1/sessions/:id/outcome` - Report purchase/abandon
- POST `/api/v1/events/batch` - Send a batch of behavioral events

4.2 The Page Unload Problem

When a user closes a tab or navigates away, the browser cancels all pending `fetch()` requests. This means any events still in the queue would be lost. This is a critical problem for analytics SDKs because the most valuable events (exit intent, final scroll position) happen right before the user leaves.

Solution: `navigator.sendBeacon()`

`sendBeacon()` is a browser API specifically designed for this use case. It guarantees the request will be sent even after the page starts unloading. The trade-off is that it is fire-and-forget (no response, no error handling). We use it as a last resort during `beforeunload` and `visibilitychange` events.

```
// Transport.sendBeacon() implementation
sendBeacon(payload: BatchPayload): boolean {
  const blob = new Blob(
    [JSON.stringify(payload)],
    { type: "application/json" }
  );
  // sdk_key as query param (no headers in sendBeacon)
  return navigator.sendBeacon(
    `${this.baseUrl}/api/v1/events/batch?sdk_key=hash`,
    blob
  );
}
```

Why Query Param for sendBeacon?

sendBeacon() does not support custom headers. So the SDK key hash is sent as a query parameter instead. The backend's get_sdk_key_header() function accepts both X-SDK-Key header and sdk_key query param, specifically to support this sendBeacon fallback.

Chapter 5

Event Queue & Batching Strategy

5.1 Why Batch Events?

Individual HTTP requests per event would create thousands of requests per session. Batching reduces network overhead, server load, and battery drain on mobile devices. The EventQueue buffers events and flushes them periodically.

Queue Configuration

- flushIntervalMs: 2000 (flush every 2 seconds)
- batchSize: 50 (flush when queue reaches 50 events)
- On page unload: flush everything via sendBeacon()

5.2 Failure Recovery

When a batch fails to send (network error, server down), the events are re-queued at the front of the queue. This ensures events are sent in order and nothing is lost during temporary outages.

```
async flush(): Promise<void> {
  const events = this.queue.splice(0, batchSize);
  try {
    await transport.sendEvents({ session_id, events });
  } catch (err) {
    // Re-queue at front on failure
    this.queue.unshift(...events);
  }
}
```

Memory Consideration

If the backend is down for an extended period, the queue grows unboundedly. In a production system, you would add a max queue size (e.g., 1000 events) and drop oldest events when exceeded. This prevents memory leaks on long-running single-page apps.

Chapter 6

Session Management & Persistence

6.1 What is a Session?

A session represents a single user visit. It starts when the SDK initializes and ends when the user navigates away. Sessions are identified by UUID v4 and stored on the backend with a 90-day expiration for GDPR compliance.

6.2 sessionStorage Persistence

When a user navigates between pages on the same site (SPA or not), we want to maintain the same session. sessionStorage persists across same-tab navigations but clears when the tab is closed. This is perfect for our use case.

```
// On init: check for existing session
const stored = sessionStorage.getItem('__conversiono_session');
if (stored && stored.merchantId === merchantId) {
  // Resume existing session (no API call needed)
  return JSON.parse(stored);
}
// Otherwise: create new session via API
const session = await transport.createSession({...});
sessionStorage.setItem('__conversiono_session', JSON.stringify(session));
```

sessionStorage vs localStorage

sessionStorage: scoped to the tab, cleared on close. Perfect for sessions.

localStorage: persists forever, shared across tabs. Would create problems with multiple tabs having the same session ID, and stale sessions persisting after the user intended to leave.

6.3 Graceful Degradation

If the backend is unreachable during session creation, the SDK still creates a local session and captures events. This means data collection continues even if the first API call fails -- events will be sent when connectivity returns.

Chapter 7

DOM Event Collectors

7.1 Architecture: Collectors as Plugins

Each collector is a standalone function that attaches DOM listeners and returns a cleanup function. This pattern makes collectors composable and easy to test independently. The SDK activates all collectors on `init()` and calls their cleanup functions on `destroy()`.

```
type Cleanup = () => void;

function collectClicks(queue: EventQueue): Cleanup {
  const handler = (e) => queue.push({ type: "click", ... });
  document.addEventListener("click", handler, { passive: true });
  return () => document.removeEventListener("click", handler);
}
```

7.2 Mouse Move Collector

Captures cursor position and calculates velocity (pixels/second). Velocity is a key ML feature: high velocity suggests scanning/uncertainty, while low velocity suggests focused reading/intent.

```
// Velocity calculation
const dt = (now - lastTime) / 1000; // seconds
const dx = ev.clientX - lastX;
const dy = ev.clientY - lastY;
const dist = Math.sqrt(dx*dx + dy*dy);
const velocity = dt > 0 ? dist / dt : 0;
```

7.3 Click Collector with Rage Click Detection

A 'rage click' is when a user clicks the same area 3+ times within 2 seconds. This is a strong signal of frustration (the button is not responding, or the user is confused about what is clickable).

```
// Rage click detection algorithm
recentClicks.push({ x, y, time: now });
// Remove clicks older than 2 seconds
while (recentClicks[0]?.time < now - 2000) recentClicks.shift();
// Count clicks within 50px radius
const nearby = recentClicks.filter(c =>
  Math.sqrt((c.x-x)**2 + (c.y-y)**2) <= 50
);
const isRageClick = nearby.length >= 3;
```

7.4 Scroll Collector

Tracks scroll direction, delta (pixels), viewport percentage, and 'scroll retreat' (user scrolled down then back up). Scroll retreat is a key behavioral signal: it indicates the user is reconsidering or re-reading content they already passed.

Chapter 8

Exit Intent & Visibility Detection

8.1 What is Exit Intent?

Exit intent is the behavioral pattern that predicts a user is about to leave. E-commerce sites use this to trigger last-chance offers (discount popups, free shipping reminders). Our SDK detects three types of exit intent:

1. Mouse leaving viewport top (desktop) - cursor moves toward browser chrome/address bar
2. Back button / history navigation - user pressed the back button
3. Tab switch (visibility hidden) - user switched to another tab

8.2 Desktop vs Mobile

On desktop, mouse position is the primary signal (mouseout event with clientY <= 0). On mobile, there is no cursor. Instead, we rely on the Page Visibility API (document.visibilityState changes to 'hidden') which fires when the user switches between tabs, or the browser goes to background.

```
// Exit intent: mouse leaves viewport top
document.addEventListener('mouseout', (e) => {
  if (e.clientY <= 0) {
    queue.push({
      type: 'exit_intent',
      metadata: { trigger: 'mouse_leave' }
    });
  }
});

// Visibility change (mobile + desktop)
document.addEventListener('visibilitychange', () => {
  if (document.visibilityState === 'hidden') {
    queue.push({ type: 'exit_intent',
      metadata: { trigger: 'tab_switch' } });
  }
});
```

iOS Safari: No beforeunload

iOS Safari does not reliably fire the 'beforeunload' event. This is why we also listen to 'visibilitychange' for flushing events. The SDK uses both listeners to maximize data capture reliability across all platforms.

Chapter 9

Backend API Endpoints (FastAPI)

9.1 Endpoint Architecture

The backend provides 4 REST endpoints for the SDK, all protected by SDK key authentication. The endpoints use async SQLAlchemy for non-blocking database operations.

POST /api/v1/sessions

Creates a new session. Generates a UUID v4 server-side, sets a 90-day expiration, and returns the session_id to the SDK.

PUT /api/v1/sessions/{id}/end

Sets ended_at timestamp. Uses UPDATE...RETURNING to atomically check ownership and update in a single query.

PUT /api/v1/sessions/{id}/outcome

Reports whether the session ended in 'purchase' or 'abandon'. This is the ground truth label for ML training.

POST /api/v1/events/batch

Bulk inserts up to 200 events. Verifies session ownership first, then uses db.add_all() for efficient batch insertion.

9.2 SDK Key Authentication

```
async def authenticate_sdk_key(db, sdk_key_hash):
    """Look up merchant by SDK key hash."""
    result = await db.execute(
        select(Merchant).where(
            Merchant.sdk_key_hash == sdk_key_hash,
            Merchant.is_active.is_(True),
        )
    )
    merchant = result.scalar_one_or_none()
    if merchant is None:
        raise HTTPException(401, "Invalid or inactive SDK key")
    return merchant
```

9.3 Pydantic Validation

Request schemas use Pydantic v2 with regex patterns and constraints. This ensures invalid data is rejected at the API

boundary before touching the database.

```
class EventItem(BaseModel):
    type: str = Field(
        ...,
        pattern=r'^(mouse_move|click|scroll|exit_intent|visibility)$'
    )
    ts: datetime
    x: float | None = None
    element_id: str | None = Field(None, max_length=128)
    metadata: dict | None = None

class EventBatchRequest(BaseModel):
    session_id: str
    events: list[EventItem] = Field(
        ..., min_length=1, max_length=200
    )
```

Chapter 10

Testing Strategy

10.1 SDK Tests (Vitest + happy-dom)

The SDK runs in a browser environment, but tests run in Node.js. happy-dom provides a lightweight DOM implementation (document, window, sessionStorage, events) so we can test collectors and session management without a real browser.

Test Coverage (74 tests)

- utils.test.ts (18 tests) - sha256, uuid4, throttle, getElementId
- transport.test.ts (10 tests) - fetch mocking, sendBeacon, error handling
- event-queue.test.ts (11 tests) - batching, periodic flush, failure recovery
- session.test.ts (13 tests) - create, resume, end, storage persistence
- collectors.test.ts (15 tests) - mouse, click, scroll, exit intent, visibility
- index.test.ts (7 tests) - init/destroy lifecycle, disabled mode

10.2 Backend Tests (Pytest + Mocks)

Backend tests use FastAPI's TestClient with dependency overrides. The database is replaced with AsyncMock, and `authenticate_sdk_key` is patched with `unittest.mock.patch`. This tests endpoint logic (routing, validation, status codes) without needing a PostgreSQL connection.

Test Coverage (24 tests)

- Session create - success, missing key, invalid body, invalid device type
- Session end - success, not found, invalid UUID, missing key
- Session outcome - purchase, abandon, invalid value, missing key
- Event batch - success, all event types, invalid type, empty, too many, session not found
- Auth - missing SDK key, header extraction
- Schema validation - element_id max length, outcome pattern, metadata dict

10.3 Key Testing Pattern: Dependency Override

```
# Override database dependency
mock_db = AsyncMock()
async def override_get_db():
    yield mock_db
app.dependency_overrides[get_db] = override_get_db

# Patch auth function (called directly, not via Depends)
@patch("app.api.sdk.authenticate_sdk_key",
       new_callable=AsyncMock,
       return_value=make_merchant())
def test_endpoint(self, mock_auth):
    response = client.post('/api/v1/sessions', ...)
    assert response.status_code == 200
```

Depends() vs Direct Calls

FastAPI's `app.dependency_overrides` only works for `Depends()` parameters. `authenticate_sdk_key()` is called directly inside endpoint functions, so we use `unittest.mock.patch()` to mock it. This is a common gotcha when testing FastAPI apps with mixed dependency injection patterns.